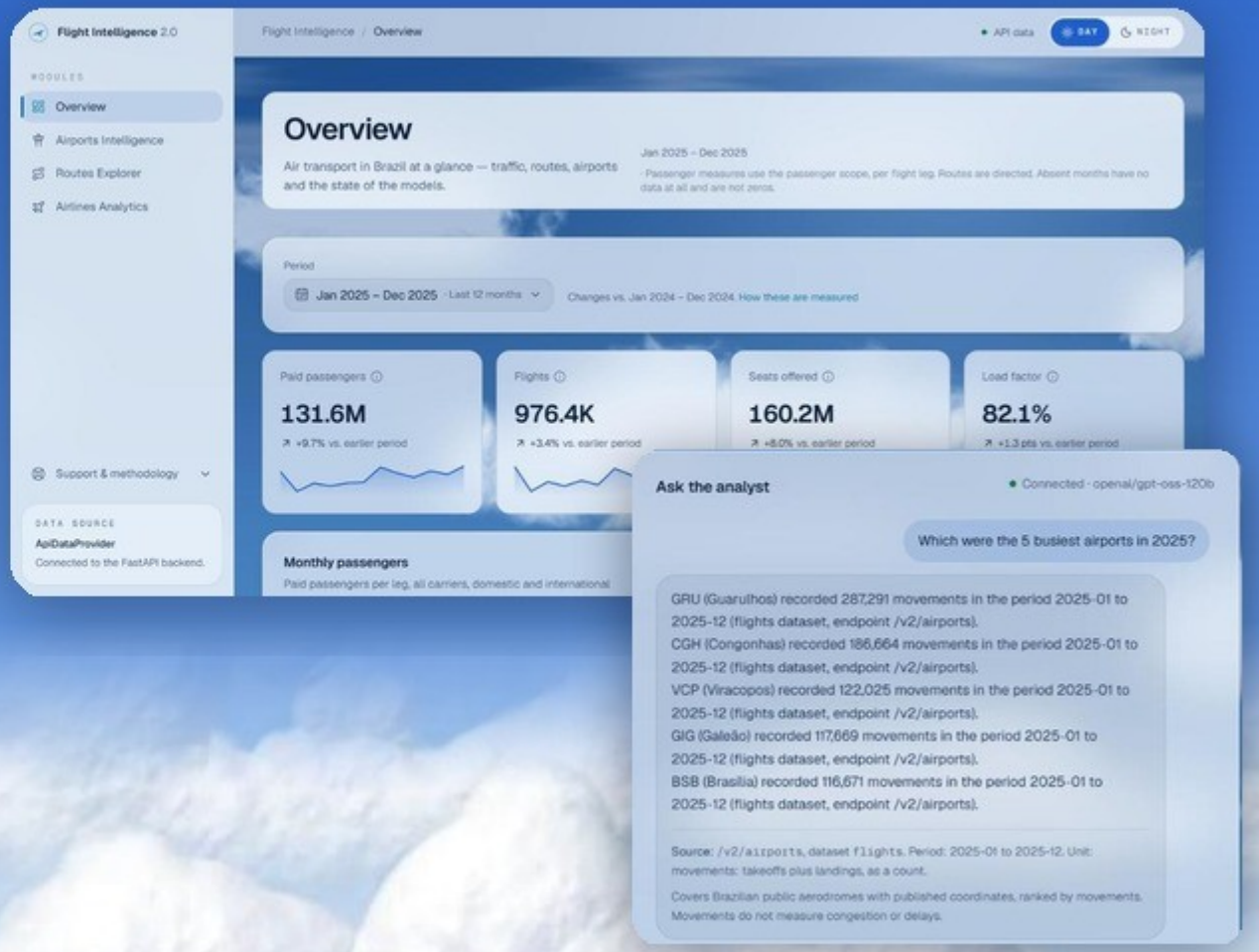
ANAC FLIGHT INTELLIGENCE

Aviação brasileira *em dados* verificáveis.

22,1 milhões de etapas de voo, 2000–2025
Modelos comparados a baselines · AI Analyst

Python · Polars · DuckDB | LightGBM | FastAPI · Next.js | Groq



anac-flight-intelligence-api-main.vercel.app

Caio Bandeira · Ciência de Dados e Machine Learning (UniCEUB)

anac-flight-intelligence-api-main.vercel.app · github.com/CaioBandeira7/anac-flight-intelligence-api-main

Problema e oportunidade

A ANAC publica todas as etapas de voo do Brasil desde 2000: cerca de 20 GB de dados abertos. Usá-los corretamente é difícil.

Contagens diferentes

Voos, voos de passageiros e movimentos (pousos + decolagens) não são a mesma coisa.

Rotas têm sentido

CGH→SDU e SDU→CGH são linhas separadas.

Ocupação não se média

Load factor = Σ passageiros pagos / Σ assentos, recalculado em cada recorte.

Meses ausentes

2015-08, 2016-02, 2018-07 e 2020-08 não existem na fonte: são lacunas, nunca zero.

Oportunidade: fazer esse trabalho uma vez, validar, e mostrar cada número com período, escopo, unidade e fonte.

Fonte dos dados e dimensão do projeto

20 GB

CSV bruto da ANAC, nunca modificado

22,1 mi

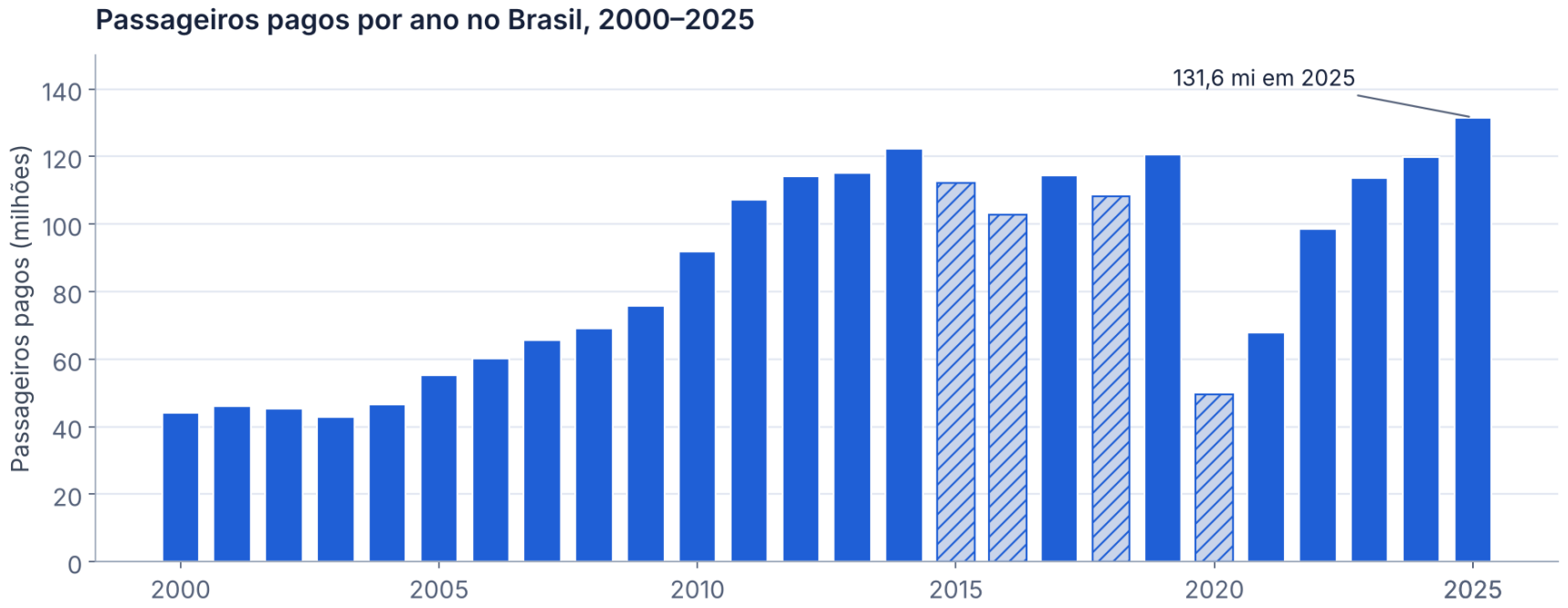
linhas lidas (22.120.241), nenhuma rejeitada

308

meses com dados, 2000-01 a 2025-12 (4 ausentes)

1,18 GB

em Parquet, ingestão completa em 8 min

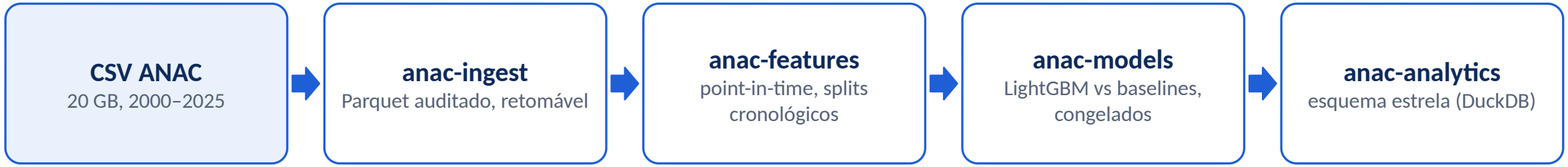


Barras hachuradas: anos com um mês ausente na fonte (2015-08, 2016-02, 2018-07, 2020-08), somados sobre 11 meses.
Fonte: API pública /v2/analytics/routes/monthly (2000-01 a 2025-12), consultada em 2026-11-08

Camada analítica: esquema estrela com 3 dimensões e 4 tabelas fato (ex.: fact_route_month com 405.134 linhas).

Fontes: docs/ingestion.md (execução completa) e /v2/analytics/coverage da API pública, consultados em 2026-11-08.

Arquitetura da solução



API FastAPI

Somente leitura · /v2/* e /v2/analytics/*

Docker no Render (imagem v3)

Dashboard Next.js 16

React 19 · TypeScript · Tailwind 4

Vercel, temas DAY/NIGHT

AI Analyst

Groq (openai/gpt-oss-120b)

5 ferramentas fixas · verificador numérico

Regras escritas para os agentes (AGENTS.md): dados nunca inventados, contrato da API espelhado em TypeScript, cada área com dono.

Detalhes: README.md (Architecture) e docs/DEPLOYMENT.md.

Engenharia e processamento dos dados

Ingestão em blocos

151 blocos, 42 MB/s, pico de 2,37 GB de RAM. Para com segurança se a memória passar do limite.

Auditoria independente

Script separado relê o CSV com DuckDB e outro parser; os totais batem com o pipeline.

Sem vazamento

Atributos point-in-time, atraso de publicação de 2 meses, fronteiras purgadas; testes embaralham o futuro.

Regras de dados

Escopo de passageiros por etapa, rotas direcionadas, meses ausentes como lacunas.

Bugs encontrados

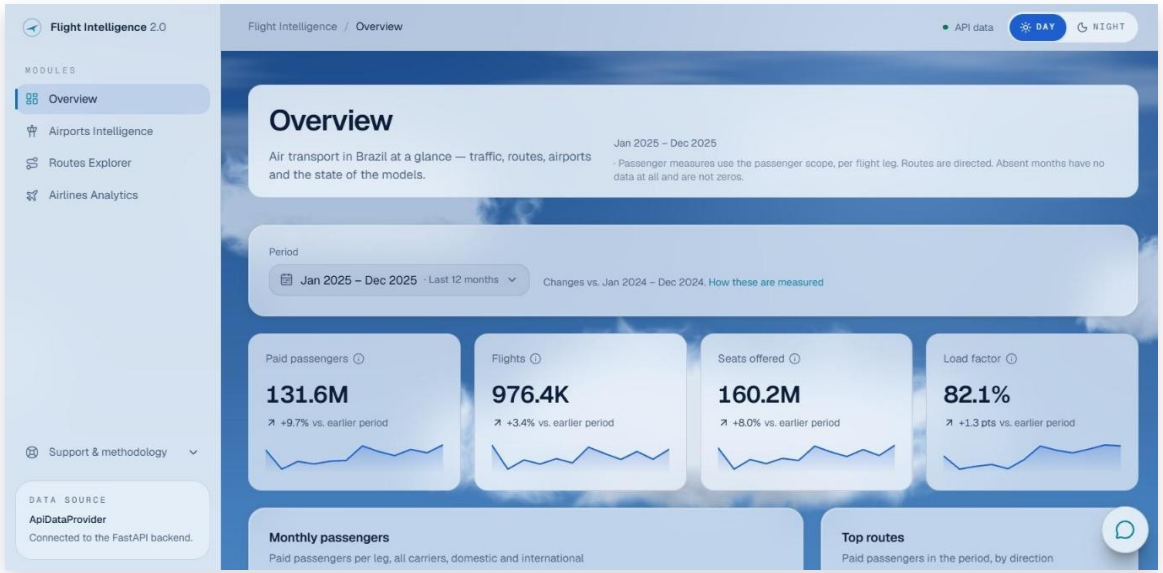
A regra original marcava 100% das linhas como internacionais; corrigido para cruzar fronteira.

Qualidade

364 testes automatizados no backend (pytest), ruff e mypy estrito, em 2026-11-08.

Fontes: docs/ingestion.md, reports/ingestion/INGESTION_REPORT.md, docs/features.md.

Dashboard analítico

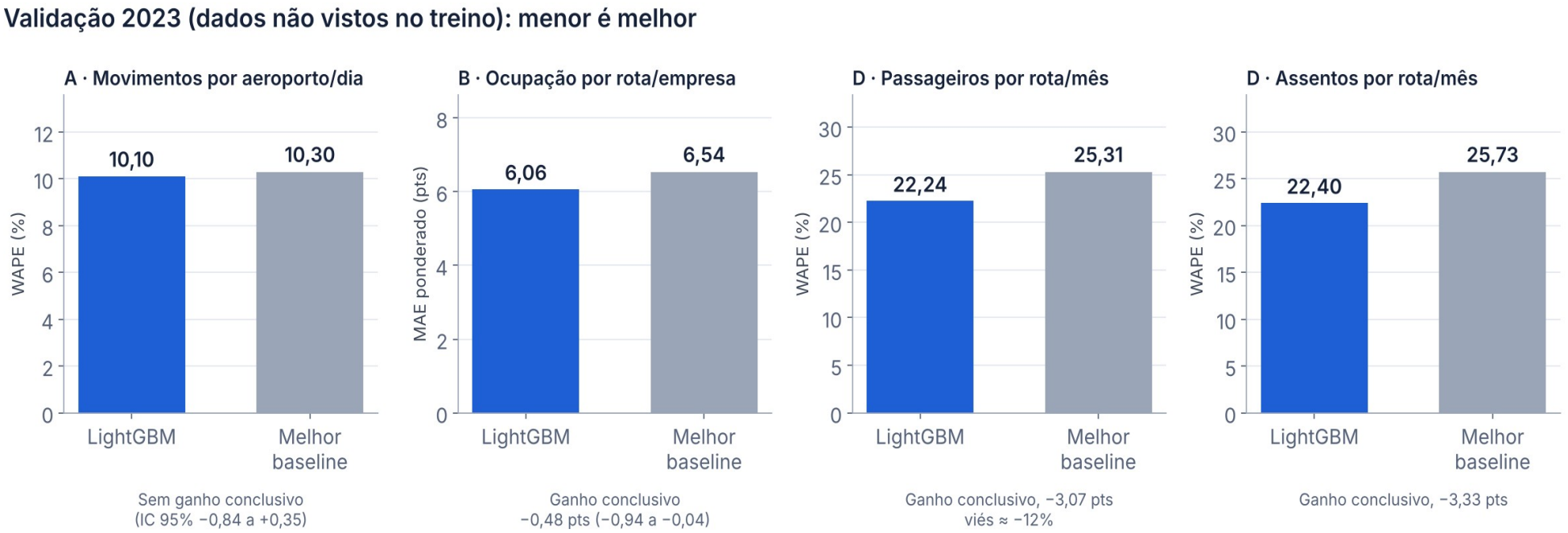




- Mercado, aeroportos, rotas e empresas para qualquer período, comparado ao mesmo período do ano anterior
- 2025: 131,6 mi de passageiros pagos (+9,7%), 976,4 mil voos, ocupação de 82,1%
- Cada tela trata carregamento, erro e vazio; meses ausentes aparecem como lacuna

Capturas reais do site público em 2026-11-08 (tema DAY).

Modelos de Machine Learning e baselines

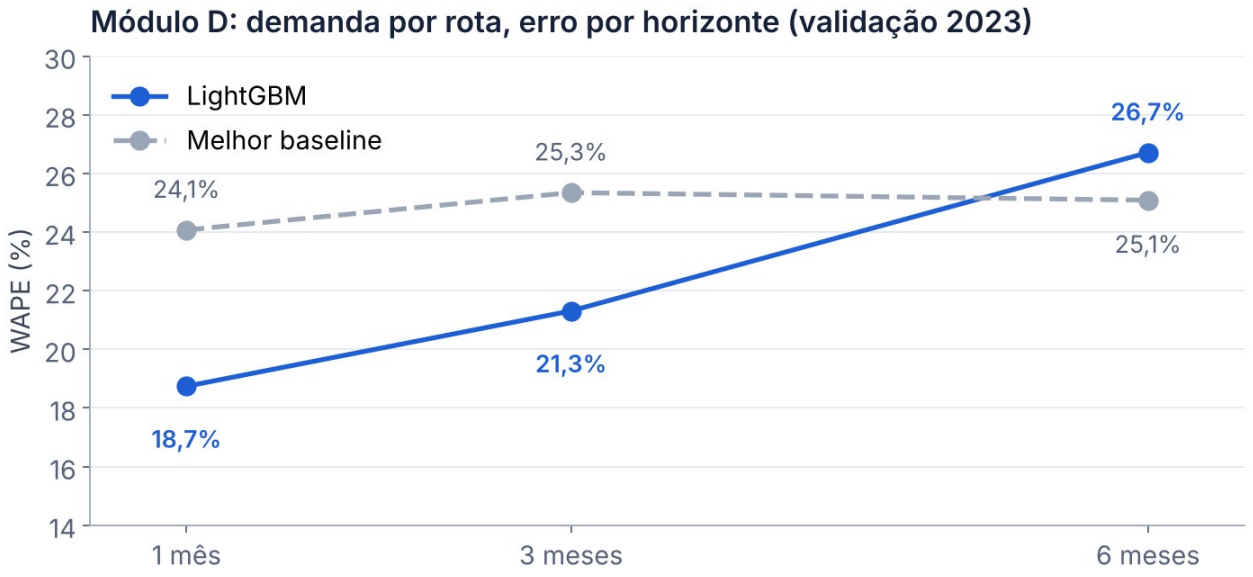


Fonte: research/ml-showcase/REPORT.md e results/analysis.json. Resultados de validação, não previsões; o teste 2024–2025 não foi executado. No módulo A o baseline é o método adotado.

- Splits cronológicos; validação em 2023, dados não vistos no treino
- Adotado só se vencer o melhor baseline com IC 95% que exclui zero (bootstrap em blocos)
- Módulo A: o baseline é mantido, porque o ganho do LightGBM não é conclusivo
- Módulo C (bagagem) não foi treinado

Fonte: research/ml-showcase/REPORT.md. Resultados de validação, não previsões; não há endpoint de previsão.

Resultados reais e limitações metodológicas



Melhor baseline em cada horizonte (mesmo mês do ano anterior, com ou sem tendência).
Ganha em 1 e 3 meses; em 6 meses fica pior.
Fonte: research/ml-showcase/results/analysis.json (tasks.d_pax.by_horizon).

O que os números sustentam

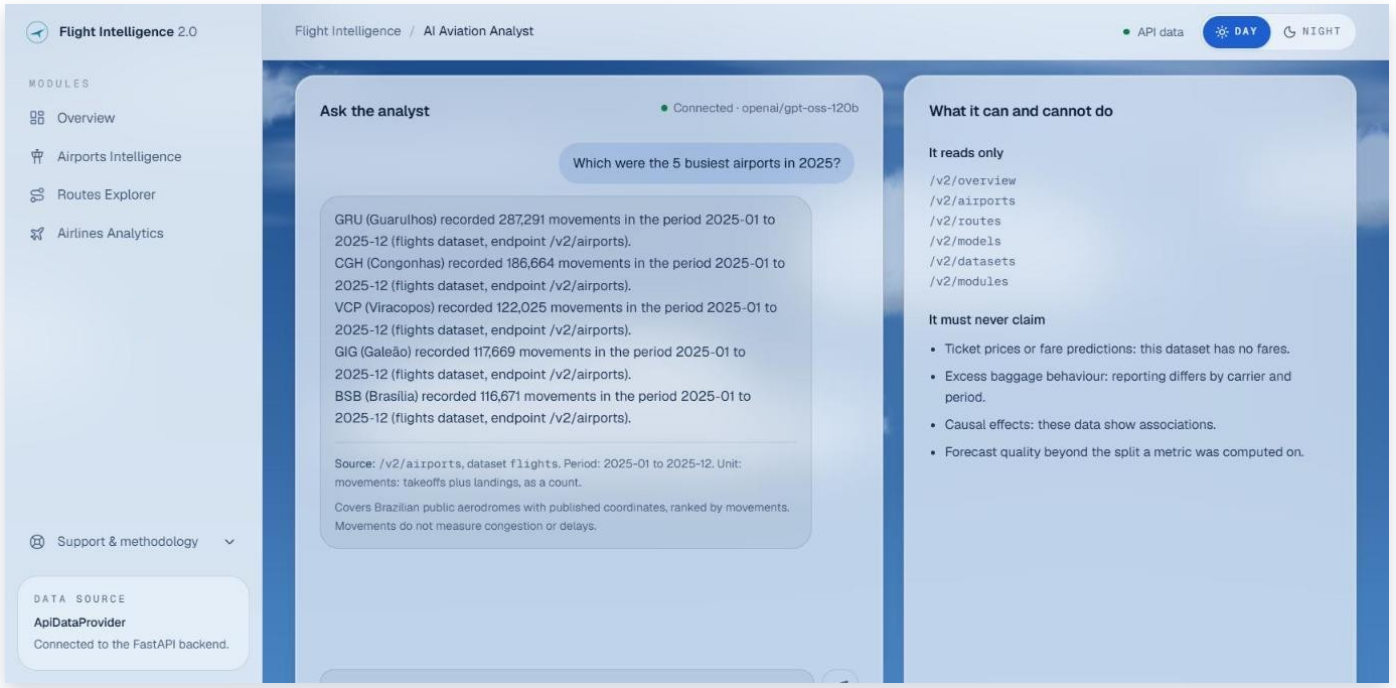
Demanda por rota, 1 mês à frente: WAPE 18,7% vs 24,1%.
Ocupação: 6,06 vs 6,54 pts.

Limitações declaradas

- Módulo D subestima cerca de 12%; em 6 meses perde do baseline
- Teste 2024–2025 selado e não executado
- Treino sem os 4 meses ausentes
- Associações, não causas; sem tarifas na base

Reprodutível: research/ml-showcase/reproduce.py --refit recalcula as métricas e reajusta os modelos em memória.

AI Analyst e integração com Groq



- Perguntas em linguagem natural respondidas só por 5 ferramentas fixas e somente leitura
- Todo número da resposta é conferido contra o resultado da ferramenta; rascunhos errados são recusados
- Fontes, período e unidade anexados pelo código, não pelo modelo
- Chave da Groq só no servidor; limite de 10 perguntas/hora por visitante e 200/dia no total

Resposta real em 2026-11-08; os 5 aeroportos e movimentos conferem com /v2/airports. O verificador é heurístico: não garante toda associação.

Demonstração pública, tecnologias e conclusões

Dashboard anac-flight-intelligence-api-main.vercel.app

AI Analyst anac-flight-intelligence-api-main.vercel.app/dashboard/analyst

ML Studio anac-flight-intelligence-api-main.vercel.app/dashboard/ml-studio

API anac-flight-intelligence-api-v2.onrender.com/docs

Python 3.12 | Polars | DuckDB | Parquet | LightGBM | FastAPI | Docker | Render | Next.js 16 | React 19 | TypeScript | Tailwind 4 | Vercel | Groq

Conclusões

- Dados abertos viram respostas corretas quando as regras de contagem são explícitas e testadas
- ML só onde vence o baseline com confiança; onde não vence, o projeto diz
- IA generativa útil quando presa a ferramentas fixas e verificação numérica

Construído com Claude Code e Codex sob regras escritas e aprovação humana de cada etapa.